

University Management System: A Comprehensive Object-Oriented Final Lab Assignment

Submission Instructions

- Submit all required Java source code files to Google Classroom.
- Submit a generated `students.txt` file containing student information.
- Submit at least **4 screenshots**:
 - One showing successful program execution.
 - One showing the polymorphic display of all university members.
 - One showing the sorted student list according to GPA.
 - One showing invalid GPA exception handling.

Purpose

This final lab focuses on a deep understanding and practical application of:

- Classes, objects, constructors, getters and setters
- Encapsulation and access modifiers
- Inheritance and method overriding
- Abstract classes
- Interfaces
- Polymorphism and dynamic method dispatch
- Upcasting, downcasting, and `instanceof`
- Custom exception handling
- Collections with `ArrayList`
- Generics
- Sorting objects using `Comparable`
- File input/output operations

Problem Description

You are to design and implement a Java application named **UniversityManagementSystem**. The system will manage students and academic staff members in a university. Each university member has common personal information, but students and academic staff have different responsibilities and behaviors.

The system must allow the programmer to create different university members, store them in collections, display their information polymorphically, calculate academic staff salaries, sort students by GPA, and write student information into a text file.

The program does not need to receive input from the user. You may create sample objects directly inside the **main** method.

Class Specifications

1) Person (Abstract Class)

Create an abstract class named **Person**.

- It must contain the following private instance variables:

```
private int id;  
private String name;
```

- It must include a constructor, getters, and setters.
- It must define the following abstract method:

```
public abstract void displayInfo();
```

2) Payable (Interface)

Create an interface named **Payable**.

- It must contain the following method:

```
double calculateSalary();
```

3) Student Class

Create a class named **Student** that extends **Person** and implements **Comparable<Student>**.

- It must contain the following private instance variables:

```
private String department;  
private double gpa;
```

- The constructor must validate GPA.
- If GPA is less than 0 or greater than 4, an **InvalidGPAException** must be thrown.
- Override the **displayInfo()** method.
- Override the **compareTo()** method so that students are sorted according to GPA in descending order.

4) AcademicStaff Class

Create a class named `AcademicStaff` that extends `Person` and implements `Payable`.

- It must contain the following protected instance variables:

```
protected String title;  
protected double baseSalary;
```

- Override the `displayInfo()` method.
- Implement the `calculateSalary()` method.
- The default salary calculation should return `baseSalary`.

5) Professor Class

Create a class named `Professor` that extends `AcademicStaff`.

- It must contain the following private instance variable:

```
private int publishedPapers;
```

- Override the `calculateSalary()` method using the following formula:

```
baseSalary + (publishedPapers * 1000)
```

- Override the `displayInfo()` method if necessary.

6) ResearchAssistant Class

Create a class named `ResearchAssistant` that extends `AcademicStaff`.

- It must contain the following private instance variable:

```
private int projects;
```

- Override the `calculateSalary()` method using the following formula:

```
baseSalary + (projects * 500)
```

- Override the `displayInfo()` method if necessary.

7) InvalidGPAException (Custom Exception)

Create a custom exception class named `InvalidGPAException`.

- It must extend `Exception`.
- Its constructor must receive an error message as a `String`.
- It must be used when an invalid GPA value is entered.

8) UniversityRepository<T> (Generic Class)

Create a generic class named `UniversityRepository<T>`.

- It must contain an `ArrayList<T>` named `items`.
- It must include the following methods:

```
void add(T item)
void remove(T item)
void displayAll()
```

Program Requirements

- Create the main class named `UniversityManagementSystem`.
- Inside the `main` method, create the following collection:

```
ArrayList<Person> people = new ArrayList<>();
```

- Add at least:
 - 2 `Student` objects
 - 2 `Professor` objects
 - 2 `ResearchAssistant` objects
- Display all people using polymorphism:

```
for (Person p : people) {
    p.displayInfo();
}
```

- Use `instanceof` and downcasting to find academic staff members and print their salaries:

```
if (p instanceof AcademicStaff) {
    AcademicStaff staff = (AcademicStaff) p;
    System.out.println(staff.calculateSalary());
}
```

- Store all students in a separate `ArrayList<Student>`.
- Sort students by GPA using `Collections.sort()`.
- Display the sorted student list.
- Write all student information into a text file named `students.txt`.
- Handle `InvalidGPAException` and file writing exceptions using `try-catch` blocks.

Technical Requirements

- You must use at least **9 Java files**:
 1. Person.java
 2. Student.java
 3. AcademicStaff.java
 4. Professor.java
 5. ResearchAssistant.java
 6. Payable.java
 7. InvalidGPAException.java
 8. UniversityRepository.java
 9. UniversityManagementSystem.java
- You must use appropriate access modifiers.
- You must override methods where necessary.
- You must use `ArrayList` for storing objects.
- You must use a generic class.
- You must use custom exception handling.
- You must use file writing operations.
- You must not use lambda expressions or streams.

Sample Output

```
=== University Members ===
Student: 1001 | Ahmet | Computer Engineering | GPA: 3.85
Student: 1002 | Ayse | Software Engineering | GPA: 3.40
Professor: 2001 | Mehmet | Prof. Dr. | Salary: 35000.0
Professor: 2002 | Elif | Assoc. Prof. | Salary: 32000.0
Research Assistant: 3001 | Ali | Res. Asst. | Salary: 22000.0
Research Assistant: 3002 | Zeynep | Res. Asst. | Salary: 21500.0

=== Academic Staff Salaries ===
Mehmet salary: 35000.0
Elif salary: 32000.0
Ali salary: 22000.0
Zeynep salary: 21500.0

=== Students Sorted by GPA ===
Ahmet - 3.85
Ayse - 3.40

Student information was written to students.txt successfully.
```

Invalid GPA Sample Output

```
Error: GPA must be between 0 and 4.
```

Rules and Constraints

- Your code must compile successfully.
- Your code must be properly indented and commented.
- You must demonstrate polymorphism clearly.
- You must not place all classes in a single file.
- You must not use lambda expressions or streams.
- Late submissions will not be accepted.

Important

Make sure your code is object-oriented, modular, readable, and follows the class structure given in this assignment.

Reflection Questions

1. Why is **Person** defined as an abstract class instead of a regular class?
2. What are the advantages of using the **Payable** interface for salary calculation?
3. How does polymorphism help when storing different object types in **ArrayList<Person>**?
4. Why is **instanceof** required before downcasting?
5. What is the benefit of using a generic repository class?
6. Why should invalid GPA values be handled with a custom exception?
7. How does implementing **Comparable<Student>** help with sorting students?